

Super Tic-Tac-Toe as a Stochastic Reinforcement Learning Environment

ZHONG Mingqian

Abstract

This project develops a stochastic super tic-tac-toe game as a reinforcement learning environment. The game is formalized as a three-level board with 96 playable squares, stochastic action realization, forfeited moves, and row, column, and diagonal winning patterns. A complete environment is implemented with action masks, stochastic move resolution, winner detection, and systematic tests.

Baseline experiments are conducted using a random agent and a one-step heuristic agent. After correcting the player-perspective logic, the heuristic agent achieves a 96.0% win rate against the random agent over 200 games. A TorchRL-based DQN pipeline is then implemented by wrapping the two-player game into a single-agent task against a fixed random opponent. The DQN agent reaches intermediate win rates around 0.66–0.70, but its final performance remains unstable. Overall, the project establishes a runnable and extensible stochastic board-game environment for reinforcement learning.

Contents

1	Phase A: Problem Formulation and Rule Interpretation	4
1.1	Problem Interpretation	4
1.2	Symbol Conventions	5
1.3	Coordinate and Indexing Scheme	5
1.4	Cross-Level Geometry and Adjacency	6
1.5	Winning Pattern Definitions	7
1.5.1	Row Patterns	7
1.5.2	Column Patterns	7
1.5.3	Diagonal Patterns	7
1.5.4	Implementation Convention for Winner Checking	7
2	Phase B: Environment Design and Implementation	8
2.1	Overall Objective of Phase B	8
2.2	Implementation Convention for Geometry	8
2.3	Geometry Construction and Static Structures	9
2.4	Neighborhood Structure and Stochastic Move Realization	9
2.5	Winning Pattern Enumeration	9
2.5.1	Row Lines	10
2.5.2	Diagonal Lines	10
2.5.3	Column Lines	10
2.5.4	Unified Winning Line Set	10
2.6	Environment Class Design	10
2.6.1	Reset	11
2.6.2	Action Mask	11
2.6.3	Step Function	11
2.7	Testing and Validation	11
2.8	Outcome of Phase B	11
3	Phase C: Baseline Agents and Matchup Evaluation	12
3.1	Objective of Phase C	12
3.2	Baseline Agent Design	12
3.2.1	Random Agent	12
3.2.2	Heuristic Agent	12

3.3	Game Execution and Evaluation Pipeline	13
3.4	Matchup Configurations	13
3.5	Experimental Results	13
3.5.1	Random vs Random	14
3.5.2	Heuristic vs Random	15
3.5.3	Heuristic vs Heuristic	15
3.6	Discussion	15
3.7	Outcome of Phase C	15
4	Phase D: TorchRL-Based DQN Training and Preliminary Learning Results	15
4.1	Objective of Phase D	16
4.2	Single-Agent Training Formulation	16
4.3	Training Environment Wrapper	16
4.4	State, Action, and Reward Design	16
4.4.1	Observation	16
4.4.2	Action	17
4.4.3	Reward	17
4.5	DQN Training Pipeline	17
4.6	Step-by-Step Training Procedure	18
4.7	Small-Scale Trial Run	18
4.8	Intermediate 100-Episode Experiment	18
4.9	Main 1000-Episode Training Experiment	18
4.10	Comparison with Baselines	19
4.11	Discussion	20
5	Phase E: Game Watching Demonstration	21
6	Phase F: Interactive Game Interface	22
7	Conclusion	23

1 Phase A: Problem Formulation and Rule Interpretation

1.1 Problem Interpretation

This project considers a two-player stochastic super tic-tac-toe game and formulates it as a reinforcement learning environment. Compared with standard tic-tac-toe, this game is substantially more complex in terms of board topology, stochastic action execution, and winning conditions. Therefore, it should not be treated as a simple deterministic board game. Instead, it is more appropriately viewed as a two-player sequential decision-making problem with a discrete state space, a discrete action space, stochastic state transitions, and adversarial interaction.

The game is played by two players in alternating turns. According to the project specification, Player 1 places noughts, while Player 2 places crosses. Player 1 moves first by default. At each discrete time step, the current player selects an empty square as the intended target position. Importantly, the selected target square is not necessarily the final realized placement. In other words, the player controls an intended action, while the environment determines the realized placement according to a fixed stochastic rule.

The stochastic action mechanism is one of the defining features of this problem. Let a denote the square selected by the current player. The environment places the mark at a with probability $1/2$, and places it at each of the eight adjacent same-level squares with probability $1/16$. Therefore, an action should not be interpreted as a deterministic placement, but rather as a noisy intended control. This requires a successful strategy to consider not only the target square itself, but also the surrounding neighborhood, boundary risks, and possible conflicts with occupied squares.

If the realized position sampled by the environment is outside the board, or if the position is already occupied, the move is forfeited. A forfeited move means that the current player loses the turn, while the board state remains unchanged. As a result, the environment contains spatially nonuniform risk: boundary and corner positions are more likely to lead to out-of-board realization, while crowded regions in the middle and late game are more likely to produce occupied-square conflicts.

The terminal conditions are also more complex than those of standard tic-tac-toe. A player wins by forming four consecutive marks in a row, four consecutive marks in a column, or five consecutive marks across a diagonal. In addition, the project statement specifies that a column win must involve at least one move in a different level. This implies that a column cannot simply be interpreted as an ordinary vertical four-in-a-column pattern within a single board plane; instead, it must reflect the multi-level structure of the board. If all playable squares are occupied and no winning condition is satisfied, the game ends in a draw.

Overall, this project formalizes the game as a two-player stochastic game. An agent must select intended target squares under adversarial interaction and stochastic action realization, and must learn robust strategies despite noisy transitions. The problem combines board-geometry modeling, stochastic action resolution, and adversarial decision-making, making it a meaningful environment-design and reinforcement-learning task.

1.2 Symbol Conventions

The following symbols are used throughout the report:

- $\mathcal{P} = \{1, -1\}$ denotes the set of players, where 1 represents Player 1 and -1 represents Player 2.
- $\mathcal{L} = \{1, 2, 3\}$ denotes the set of board levels.
- $\mathcal{B}_\ell$ denotes the set of blocks in level ℓ , with

$$|\mathcal{B}_1| = 1, \quad |\mathcal{B}_2| = 2, \quad |\mathcal{B}_3| = 3.$$

- $(level, block, row, col)$ denotes the logical coordinate of a playable square.
- $(level, gr, gc)$ denotes the level-wise geometric coordinate of a playable square.
- $index \in \{0, 1, \dots, 95\}$ denotes the one-dimensional action index associated with a playable square.
- The board state is denoted by a vector or array s , where each playable square takes values in $\{-1, 0, 1\}$, representing Player 2, empty, and Player 1, respectively.
- The intended action selected by a player is denoted by a , while the realized placement is denoted by z .

1.3 Coordinate and Indexing Scheme

To represent the board state unambiguously and to connect it naturally with a discrete reinforcement learning action space, each playable square is assigned both a logical coordinate and a one-dimensional action index. This dual representation preserves the hierarchical board structure while supporting state encoding, action masking, and winning-line enumeration.

The board consists of six 4×4 blocks arranged into three levels. Level 1 contains one 4×4 block, Level 2 contains two horizontally adjacent 4×4 blocks, and Level 3 contains three horizontally adjacent 4×4 blocks. Therefore, the total number of playable squares is

$$6 \times 4 \times 4 = 96.$$

Thus, the action space can be naturally represented as a discrete set of size 96, where each action corresponds to selecting a target playable square.

A square is logically represented by

$$(level, block, row, col),$$

where $level \in \{1, 2, 3\}$, $block$ denotes the block index within a level, and $row, col \in \{0, 1, 2, 3\}$ denote the local row and column coordinates inside a 4×4 block. The admissible block indices are

$$block \in \begin{cases} \{0\}, & level = 1, \\ \{0, 1\}, & level = 2, \\ \{0, 1, 2\}, & level = 3. \end{cases}$$

Each playable square is further mapped to a unique one-dimensional index

$$index \in \{0, 1, \dots, 95\}.$$

The index is assigned in lexicographic order over $(level, block, row, col)$. With level offsets

$$offset(1) = 0, \quad offset(2) = 16, \quad offset(3) = 48,$$

the index of a square is defined as

$$index = offset(level) + 16 \cdot block + 4 \cdot row + col.$$

This indexing scheme is reversible. Given an index, one can identify the level, subtract the corresponding offset, and recover the block, row, and column. Therefore, the implementation maintains mappings between indices and logical coordinates.

1.4 Cross-Level Geometry and Adjacency

The logical coordinate is sufficient for structural identification, but it is not enough to determine geometric continuity. Winning patterns depend on spatial continuity across blocks and potentially across levels. Therefore, each playable square is also embedded into a level-wise geometric coordinate

$$(level, gr, gc),$$

where gr is the global row coordinate within the level and gc is the global column coordinate within the level.

Since each block has four rows,

$$gr = row, \quad gr \in \{0, 1, 2, 3\}.$$

The global column coordinate is defined by horizontally concatenating blocks:

$$gc = 4 \cdot block + col.$$

Thus, the column ranges are

$$\text{Level 1 : } gc \in \{0, 1, 2, 3\},$$

$$\text{Level 2 : } gc \in \{0, 1, \dots, 7\},$$

$$\text{Level 3 : } gc \in \{0, 1, \dots, 11\}.$$

This creates a left-aligned geometric embedding of the board.

The same-level adjacent squares are defined using the Moore neighborhood. For a selected square $(level, gr, gc)$, its eight adjacent candidate positions are

$$\{(level, gr + \Delta r, gc + \Delta c) : \Delta r, \Delta c \in \{-1, 0, 1\}, (\Delta r, \Delta c) \neq (0, 0)\}.$$

Adjacency is defined only within the same level. If a candidate position is outside the playable region of the current level, it is treated as outside the board. If the stochastic realization samples such a position, the move is forfeited rather than being redistributed to valid positions.

1.5 Winning Pattern Definitions

The winning patterns are defined using geometric continuity rather than index adjacency. A sequence of squares forms a valid winning pattern only if it is geometrically continuous under the coordinate system described above.

1.5.1 Row Patterns

A row win is defined as four horizontally consecutive playable squares within the same level. Formally, if a player occupies

$$(level, gr, gc), \quad (level, gr, gc + 1), \quad (level, gr, gc + 2), \quad (level, gr, gc + 3),$$

and all four squares are valid, then the player forms a row win. This definition allows row patterns to cross block boundaries as long as the squares remain horizontally continuous in the level-wise geometric coordinate.

1.5.2 Column Patterns

A column win requires four geometrically continuous vertical squares and must involve at least two different levels. This condition reflects the project statement that a column win must include a move in a different level. Therefore, ordinary vertical four-in-a-column patterns within a single level are not treated as column wins. In the implementation, candidate column patterns are enumerated in a stacked display coordinate system and are retained only if they contain four valid squares and involve at least two distinct levels.

1.5.3 Diagonal Patterns

A diagonal win requires five geometrically consecutive playable squares along a diagonal direction. Since a single 4×4 block contains diagonals of length at most four, a valid length-five diagonal must extend across a larger geometric region, potentially crossing block boundaries. The two basic diagonal directions are

$$(\Delta r, \Delta c) = (1, 1) \quad \text{and} \quad (\Delta r, \Delta c) = (1, -1).$$

A player wins diagonally if the player occupies five valid squares that form a continuous sequence in one of these directions.

1.5.4 Implementation Convention for Winner Checking

In the implementation, all legal winning lines are pre-enumerated during initialization. The set of winning lines is denoted by

$$\mathcal{W} = \{W_1, W_2, \dots, W_M\},$$

where each W_i is a sequence of indices corresponding to a valid row, column, or diagonal pattern. During gameplay, after each effective placement, the environment checks whether

there exists a line $W_i \in \mathcal{W}$ such that all positions in W_i are occupied by the same player. If so, that player is declared the winner.

This pre-enumeration strategy separates rule specification from runtime checking, simplifies debugging, and allows the correctness of winning patterns to be verified through tests.

2 Phase B: Environment Design and Implementation

The goal of Phase B is to transform the rule interpretation and geometric formalization into a runnable, testable reinforcement learning environment. This phase implements the static board geometry, action and state spaces, stochastic move realization, winner detection, draw detection, and systematic validation tests. The resulting **SuperTicTacToeEnv** serves as the foundation for baseline agents and DQN training.

2.1 Overall Objective of Phase B

Phase A focuses on rule clarification and formal specification, while Phase B turns the specification into executable code. The main objectives are:

1. to construct static geometric structures and index mappings for all playable squares;
2. to implement core environment interfaces, including `reset()`, `step(action)`, legality checking, observation construction, and action masking;
3. to implement stochastic move realization according to the required probabilities;
4. to implement winner and draw detection and validate them through targeted tests.

2.2 Implementation Convention for Geometry

The game rules are implemented in `geometry.py`. The module does not redefine the rules; rather, it converts the formal specification into static data structures that can be queried efficiently during environment execution.

Each square maintains three coordinate representations: the logical coordinate (*level*, *block*, *row*, *col*), the level-wise geometric coordinate (*level*, *gr*, *gc*), and the stacked display coordinate (*br*, *bc*). Logical coordinates recover the hierarchical block structure; level-wise geometric coordinates support same-level neighborhoods and horizontal continuity; stacked display coordinates support length-five diagonals and cross-level column patterns.

In the implementation, the following static structures are constructed:

- `index_to_lbrc` and `lbrc_to_index`;
- `index_to_geom` and `geom_to_index`;
- `index_to_display` and `display_to_index`;
- `neighbors`;
- `row_lines`, `diag_lines`, and `column_lines`;
- `winning_lines`.

The final implementation contains 96 playable squares, 60 row lines, 52 diagonal lines, and 36 column lines, giving a total of 148 winning lines. The environment’s `check_winner()`

function is entirely based on this pre-enumerated set.

2.3 Geometry Construction and Static Structures

The `geometry.py` module is responsible for constructing the static geometric information required by the environment. It provides index mappings, coordinate conversions, neighborhood tables, and winning-line collections. These structures allow the environment to switch between different coordinate systems during action execution, neighbor lookup, winning-line enumeration, and rendering.

2.4 Neighborhood Structure and Stochastic Move Realization

The stochastic move mechanism is one of the most important features of the environment. A player first selects an intended action, and the environment then samples the realized position. Specifically,

$$P(\text{realized action} = a) = \frac{1}{2},$$

where a is the intended target square. For each of the eight same-level neighbors n_i ,

$$P(\text{realized action} = n_i) = \frac{1}{16}, \quad i = 1, \dots, 8.$$

To support this mechanism, `geometry.py` precomputes a `neighbors` table. Each entry maps an index to a list of eight neighbor candidates. If a neighbor direction is outside the board, the corresponding entry is set to `None`. The function `resolve_action()` then performs the following steps:

1. construct the candidate list consisting of the original target and its eight same-level neighbors;
2. sample one candidate using the fixed probability distribution;
3. if the sampled candidate is `None`, forfeit the move;
4. if the sampled candidate is occupied, forfeit the move;
5. otherwise, place the current player's mark at the realized position.

This implementation preserves the original probability distribution and does not redistribute probability mass from invalid outcomes. For example, a corner square has five out-of-board adjacent directions, so the out-of-board forfeiture probability is 5/16.

2.5 Winning Pattern Enumeration

After constructing the static coordinate mappings, all legal winning patterns are pre-enumerated. The environment separately generates `row_lines`, `diag_lines`, and `column_lines`, and then merges them into `winning_lines`. During gameplay, `check_winner()` only needs to iterate over this set and check whether any winning line is fully occupied by one player.

2.5.1 Row Lines

`row_lines` are generated using the level-wise geometric coordinate. For each row in each level, a sliding window of length four is used to enumerate horizontally continuous playable squares. The final implementation generates 60 row lines:

$$4 \times (1 + 5 + 9) = 60.$$

2.5.2 Diagonal Lines

`diag_lines` are generated using the stacked display coordinate. The algorithm enumerates length-five candidates in both diagonal directions and retains only those whose positions all correspond to valid playable squares. The final implementation generates 52 diagonal lines.

2.5.3 Column Lines

`column_lines` are also generated in the stacked display coordinate. The algorithm first enumerates vertically consecutive length-four candidates and then keeps only those involving at least two distinct levels. This excludes ordinary within-level vertical lines. The final implementation generates 36 column lines.

2.5.4 Unified Winning Line Set

The final number of winning lines is

$$|\text{winning_lines}| = |\text{row_lines}| + |\text{diag_lines}| + |\text{column_lines}| = 60 + 52 + 36 = 148.$$

This design keeps the runtime environment logic simple: winner checking becomes a fixed traversal over a static list of valid patterns.

2.6 Environment Class Design

The main environment class is `SuperTicTacToeEnv`. It uses a flat board array of length 96:

$$\text{board}[i] \in \{-1, 0, 1\},$$

where -1 represents Player 2, 0 represents an empty square, and 1 represents Player 1. The environment maintains the following variables:

- `current_player`;
- `done`;
- `winner`;
- `move_count`;
- `last_action`;
- `last_realized_action`;
- `last_move_forfeited`.

2.6.1 Reset

The `reset()` function clears the board, sets the current player to Player 1, resets all terminal and bookkeeping variables, and returns the initial observation. The observation consists of the 96-dimensional board state and one additional dimension for the current player, giving a total dimension of 97.

2.6.2 Action Mask

The environment implements `get_action_mask()`, which returns a Boolean vector of length 96. Empty squares are marked as `True`, while occupied squares are marked as `False`. This mask is used later by the DQN agent to avoid illegal actions.

2.6.3 Step Function

The `step(action)` function is the core environment interface. It checks action legality, resolves the stochastic realization, handles forfeited moves, updates the board if a valid placement occurs, checks for a winner or draw, switches the current player if necessary, and returns the new observation and auxiliary information. This design explicitly separates the intended action from the realized action, accurately reflecting the stochastic execution rule.

2.7 Testing and Validation

A manual test script, `run_tests.py`, was developed to validate the environment. The tests cover:

- reset behavior;
- action masks;
- legal and illegal actions;
- successful and forfeited steps;
- stochastic realization probabilities;
- boundary forfeiture probability;
- row, diagonal, and column winner detection;
- false-positive cases for incomplete winning lines;
- observation and bookkeeping consistency.

All tests passed, indicating that the environment is not only executable but also consistent with the formal rule interpretation.

2.8 Outcome of Phase B

Phase B successfully transforms the formal rule specification into a verified environment. The main outputs are:

- a complete board geometry model with coordinate mappings;
- the stochastic move and forfeiture mechanism;

- row, diagonal, and column winning-line enumeration;
- the `SuperTicTacToeEnv` environment class;
- systematic tests covering the key environment logic.

At the end of Phase B, the project has a validated stochastic two-player board-game environment ready for baseline evaluation and reinforcement learning training.

3 Phase C: Baseline Agents and Matchup Evaluation

After completing the environment implementation, the project proceeds to baseline agent construction and matchup evaluation. The purpose of this phase is not to train an RL agent immediately, but to first establish simple and interpretable baselines. These baselines help verify whether the environment behaves reasonably in complete games and provide comparison points for later DQN training.

3.1 Objective of Phase C

The objectives of Phase C are:

1. to verify that complete games terminate properly and exhibit reasonable dynamics;
2. to construct interpretable baseline agents;
3. to establish a standardized evaluation pipeline for later RL experiments.

3.2 Baseline Agent Design

3.2.1 Random Agent

The `RandomAgent` uniformly samples from all legal actions. If the legal action set in state s is denoted by $\mathcal{A}_{\text{legal}}(s)$, then

$$\pi_{\text{rand}}(a \mid s) = \begin{cases} \frac{1}{|\mathcal{A}_{\text{legal}}(s)|}, & a \in \mathcal{A}_{\text{legal}}(s), \\ 0, & \text{otherwise.} \end{cases}$$

This agent serves as the minimal performance baseline.

3.2.2 Heuristic Agent

The `HeuristicAgent` uses a simple one-step deterministic approximation. Its policy is:

1. if the current player has an immediate winning move, take it;
2. otherwise, if the opponent has an immediate winning threat, block it;
3. otherwise, choose a random legal action.

The heuristic agent has been corrected to reason from the perspective of the current player, rather than assuming a fixed player identity. This ensures that it behaves consistently whether it plays as Player 1 or Player 2. Since the actual environment is stochastic, this heuristic is not optimal; however, it is a strong and interpretable baseline.

3.3 Game Execution and Evaluation Pipeline

The function `play_one_game()` runs a complete game between two agents. It resets the environment, selects the appropriate agent according to `current_player`, executes actions until termination, and returns summary statistics such as winner, draw status, move count, total turns, and optional game history.

The function `evaluate_matchup()` repeats this process for N games and reports:

- Player 1 win count and win rate;
- Player 2 win count and win rate;
- draw count and draw rate;
- average move count;
- average number of turns.

Both `move_count` and total turns are recorded because forfeited moves may increase the number of turns without increasing the number of successful placements.

3.4 Matchup Configurations

Three baseline matchups are evaluated:

1. **Random vs Random;**
2. **Heuristic vs Random;**
3. **Heuristic vs Heuristic.**

Each matchup is evaluated over 200 games with fixed seed settings for reproducibility.

3.5 Experimental Results

The baseline results are shown in Table 1.

Table 1: Baseline matchup results in Phase C					
Matchup	P1 Win	P2 Win	Draw	Avg Move Count	Avg Turns
Random vs Random	0.545	0.455	0.000	50.77	68.00
Heuristic vs Random	0.960	0.040	0.000	31.21	39.55
Heuristic vs Heuristic	0.565	0.435	0.000	31.91	40.49

To visualize these results, Figure 1 compares win rates across the three matchups, while Figure 2 compares average game lengths.

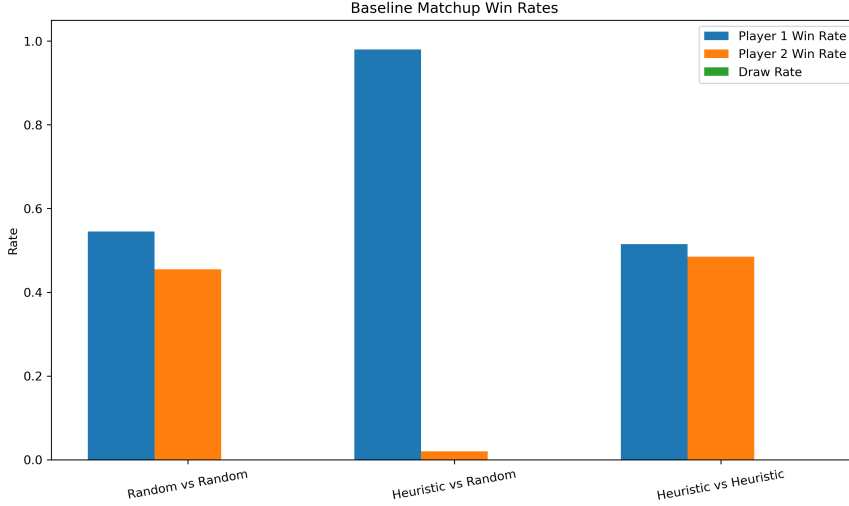


Figure 1: Win rate comparison for different baseline matchups.

Figure 1 shows that Random vs Random is relatively balanced, with a mild first-player advantage. In Heuristic vs Random, the heuristic agent achieves a win rate of 96.0%, showing that even a simple one-step heuristic can strongly outperform random play. In Heuristic vs Heuristic, Player 1 wins 56.5% of the games, indicating a moderate first-player advantage but no severe imbalance.

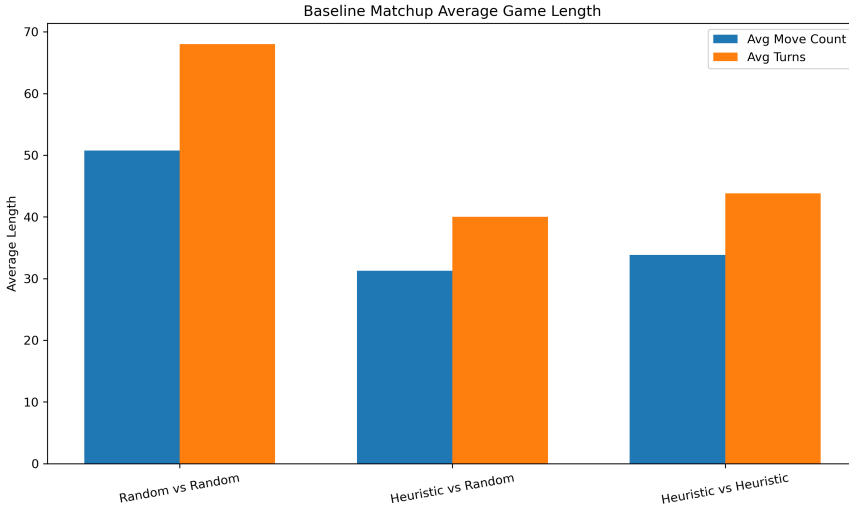


Figure 2: Average game length comparison for different baseline matchups.

Figure 2 shows that Random vs Random produces longer games, with an average move count of 50.77 and an average turn count of 68.00. In contrast, Heuristic vs Random and Heuristic vs Heuristic are much shorter, indicating that heuristic play accelerates the formation or prevention of decisive patterns.

3.5.1 Random vs Random

In Random vs Random, Player 1 wins 54.5% of games and Player 2 wins 45.5%. No draws occur. This suggests a mild first-player advantage, which is reasonable for a turn-based

board game. The average turn count is larger than the average move count because some moves are forfeited.

3.5.2 Heuristic vs Random

In Heuristic vs Random, the heuristic player wins 96.0% of the games. This demonstrates that the environment can clearly distinguish between strategies of different quality, and that stochastic action realization does not completely obscure strategic advantage.

3.5.3 Heuristic vs Heuristic

When both players use the heuristic policy, Player 1 wins 56.5% of the games and Player 2 wins 43.5%. This indicates a more visible first-player advantage than in Random vs Random, but the result is still within a reasonable range. The shorter game length suggests that heuristic play leads to decisive board configurations more quickly.

3.6 Discussion

The baseline experiments serve two purposes. First, they provide direct comparison points for the later DQN agent. Second, they dynamically validate the environment beyond single-step unit tests. The results show that games terminate reliably, strategy quality is reflected in outcome statistics, and the stochastic mechanism does not prevent meaningful strategic differentiation.

The heuristic policy is still shallow: it only performs one-step deterministic evaluation and does not reason over long-term opportunities or the full stochastic realization distribution. Therefore, although it is much stronger than random play, it should not be treated as an optimal policy. This leaves room for learning-based agents to develop more robust strategies.

3.7 Outcome of Phase C

Phase C completes:

- the implementation of `RandomAgent` and `HeuristicAgent`;
- the correction of the heuristic player’s perspective logic;
- the implementation of one-game and multi-game evaluation pipelines;
- three 200-game baseline matchups;
- a set of clear performance baselines for reinforcement learning.

4 Phase D: TorchRL-Based DQN Training and Preliminary Learning Results

After constructing and evaluating baseline agents, the project trains a learning-based agent. Unlike Phase C, where strategies are manually designed, Phase D trains a value-

based agent through environment interaction. The original two-player game is wrapped into a single-agent task where the learning agent plays as Player 1 against a fixed random opponent.

4.1 Objective of Phase D

The objectives of Phase D are:

1. to wrap the stochastic two-player environment into a single-agent training environment;
2. to implement and validate a DQN/Double DQN-style training pipeline;
3. to compare the learned policy against the baseline results from Phase C.

4.2 Single-Agent Training Formulation

Although the original game is a two-player adversarial game, this phase uses a fixed-opponent formulation. The learning agent always plays as Player 1, while the opponent is a fixed `RandomAgent`. Each external call to `step(action)` first executes the learning agent’s move, then automatically executes one random opponent move if the game has not yet terminated. The resulting state is then returned to the learning agent.

This formulation converts the alternating two-player game into a standard single-agent decision problem from the learner’s perspective.

4.3 Training Environment Wrapper

The wrapper environment, `SingleAgentVsRandomEnv`, is built on top of `SuperTicTacToeEnv`. Its interaction logic is:

1. reset the base environment and ensure that the returned state is Player 1’s turn;
2. execute the learning agent’s intended action;
3. if the game terminates, return the terminal transition;
4. otherwise, execute a random opponent action;
5. return the next state from the learning agent’s perspective.

4.4 State, Action, and Reward Design

4.4.1 Observation

The observation is a 97-dimensional vector. The first 96 dimensions represent the flattened board state, and the final dimension stores the current player indicator. Since the wrapper returns states only when it is Player 1’s turn, the last dimension is usually constant, but it is retained for consistency with the base environment.

4.4.2 Action

The action space is discrete with 96 possible intended target squares:

$$a \in \{0, 1, \dots, 95\}.$$

An action mask is provided to indicate legal empty squares. The DQN policy uses this mask to avoid occupied positions.

4.4.3 Reward

The reward is defined from Player 1’s perspective:

$$r = \begin{cases} +1, & \text{if Player 1 wins,} \\ -1, & \text{if Player 1 loses,} \\ 0, & \text{if the game draws or the transition is non-terminal.} \end{cases}$$

This terminal-only reward preserves the zero-sum nature of the game but also makes the learning problem sparse.

4.5 DQN Training Pipeline

The project implements a DQN/Double DQN-style training pipeline and uses TorchRL data structures for replay storage. The pipeline consists of:

- a feedforward Q-network `QNetwork`;
- a target network `target_net`;
- a TorchRL `TensorDictReplayBuffer`;
- `LazyTensorStorage` for replay storage;
- epsilon-greedy exploration;
- action masking for legal-action selection;
- periodic target-network synchronization and evaluation.

The Q-network maps the 97-dimensional observation to 96 Q-values, one for each possible action. During action selection, invalid actions are assigned extremely small Q-values so that both random exploration and greedy exploitation operate only over legal actions.

Each transition is represented as

$$(s, a, r, s', done, mask, mask').$$

In the implementation, each transition is packed into a TorchRL `TensorDict` and stored in `TensorDictReplayBuffer`. The replay buffer uses `LazyTensorStorage`, which provides a unified tensor-dictionary representation for observations, actions, rewards, done flags, and action masks. This design is closer to the TorchRL data-management style than a hand-written list-based replay buffer.

Once the replay buffer reaches the minimum required size, mini-batches are sampled and Bellman updates are performed. The target value is computed using a Double DQN-style update: the online network selects the best legal next action under the next-state action mask, while the target network estimates the value of that selected action. This reduces overestimation bias compared with vanilla DQN.

4.6 Step-by-Step Training Procedure

Before the final 1000-episode experiment, training was scaled gradually:

1. 20 episodes to verify execution, replay growth, target-network updates, and checkpoint saving;
2. 100 episodes to observe early learning behavior;
3. 1000 episodes as the main training experiment, with evaluation every 100 episodes.

4.7 Small-Scale Trial Run

In the 20-episode trial run, the training pipeline executed successfully. Evaluations at episodes 10 and 20 both produced:

$$\text{win rate} = 0.70, \quad \text{loss rate} = 0.30, \quad \text{avg return} = 0.40.$$

Although this result is based on a very small number of training and evaluation episodes, it confirms that the environment wrapper, Q-network, replay buffer, target network, and evaluation logic are functional.

4.8 Intermediate 100-Episode Experiment

In the 100-episode experiment, the evaluation results were:

- Episode 20: win = 0.65, loss = 0.35;
- Episode 40: win = 0.70, loss = 0.30;
- Episode 60: win = 0.65, loss = 0.35;
- Episode 80: win = 0.55, loss = 0.45;
- Episode 100: win = 0.55, loss = 0.45.

These results suggest that the agent can capture some useful learning signal early in training, but performance remains unstable.

4.9 Main 1000-Episode Training Experiment

In the main experiment, the DQN agent was trained for 1000 episodes and evaluated every 100 episodes. The results are shown in Table 2.

Table 2: Evaluation results during 1000-episode DQN training

Episode	Win Rate	Loss Rate	Avg Return
100	0.62	0.38	0.24
200	0.66	0.34	0.32
300	0.60	0.40	0.20
400	0.66	0.34	0.32
500	0.58	0.42	0.16
600	0.60	0.40	0.20
700	0.54	0.46	0.08
800	0.70	0.30	0.40
900	0.64	0.36	0.28
1000	0.54	0.46	0.08

The results show three main patterns. First, the agent achieves win rates above the random first-player baseline at several intermediate checkpoints, especially around episodes 200, 400, and 800. Second, the performance is highly non-monotonic and fluctuates substantially. Third, the final win rate at episode 1000 is 0.54, which is close to the Random vs Random first-player baseline of 0.545. Therefore, the final policy does not stably outperform the random first-player baseline.

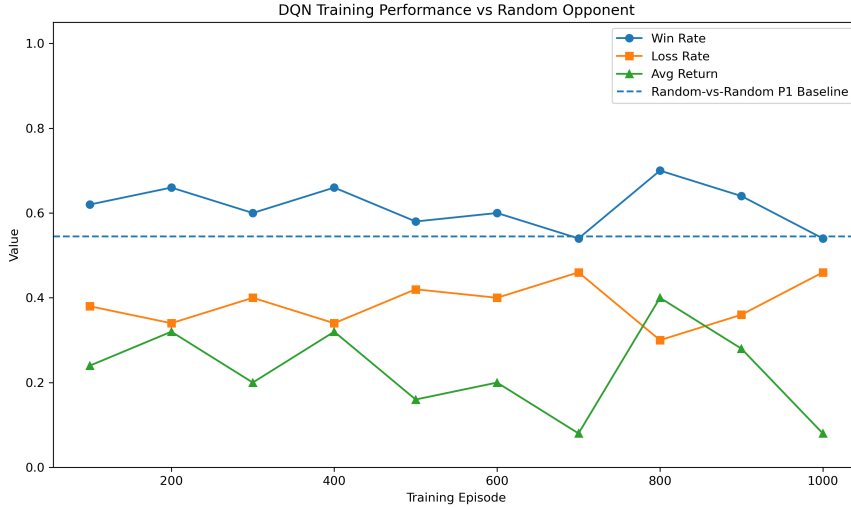


Figure 3: Training performance of the DQN agent against the random opponent.

The main conclusion is not that DQN has learned a strong strategy, but that the DQN training framework has been successfully established and is capable of capturing some useful strategy signals during training.

4.10 Comparison with Baselines

Figure 4 compares the DQN results with the baseline strategies.

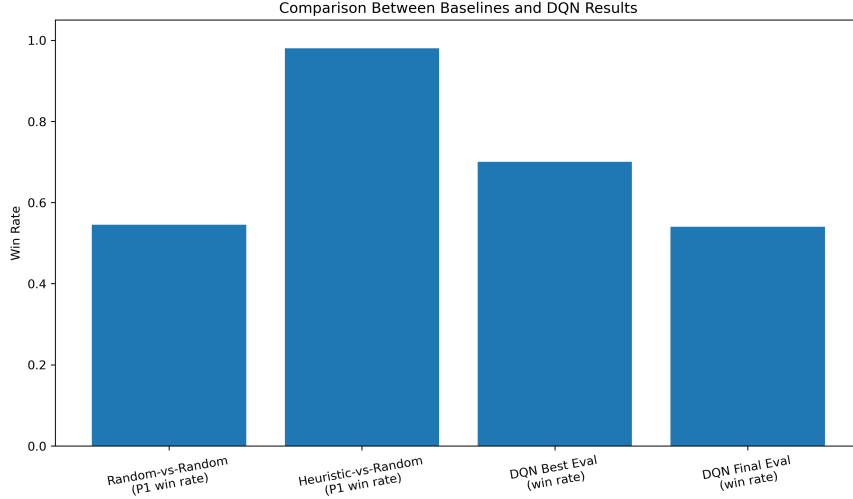


Figure 4: Comparison between baseline strategies and DQN results.

Compared with the Random vs Random first-player baseline, the DQN agent performs better at some intermediate checkpoints. This indicates that learning-based methods can extract useful information from the environment. However, compared with the **HeuristicAgent**, the current DQN policy remains much weaker. The heuristic policy achieves a 0.96 win rate against the random agent, while the best DQN evaluation reaches approximately 0.70. This gap is reasonable because the heuristic agent explicitly uses the winner-detection logic for one-step tactical decisions, whereas DQN must learn value structure from sparse rewards and stochastic transitions.

4.11 Discussion

The most important achievement of Phase D is that the project successfully connects the custom stochastic board-game environment to an end-to-end learning pipeline. The learning agent can interact with the environment, collect transitions, store them in a TorchRL replay buffer, and update a Q-network.

At the same time, the results reveal several challenges:

1. The action space is large, with 96 discrete actions.
2. The environment is stochastic, so intended actions are not always realized.
3. The reward is sparse, since meaningful feedback is mostly terminal.
4. The board geometry and winning patterns are more complex than those in standard small board games.

Thus, the observed “learned but unstable” behavior is understandable. It does not imply that the training framework is invalid; rather, it suggests that the task requires longer training, more careful hyperparameter tuning, or more suitable board-game learning methods.

5 Phase E: Game Watching Demonstration

In addition to numerical evaluation, this project also implements a simple terminal-based game watching script, `watch_game.py`. The purpose of this script is to make the behavior of the environment and agents directly observable. It can be used not only for baseline agents, but also for any agent that implements the same action-selection interface, including a trained DQN agent after loading its policy.

During execution, `watch_game.py` prints the current player, the agent being used, the intended action, the realized action, whether the move is forfeited, the current winner status, and the rendered board after each step. Therefore, this script provides an intuitive way to inspect several important mechanisms of the environment, including stochastic action realization, turn switching, forfeited moves, board updates, and terminal winner detection.

The following example shows the final state of one game between `HeuristicAgent` and `RandomAgent`. In this demonstration, Player 1 uses `HeuristicAgent` and is represented by 0, while Player 2 uses `RandomAgent` and is represented by X. The game terminates after 27 total turns, with 25 successful placements, and Player 1 wins.

```
Level 1
. . . .
. X 0 .
. 0 X .
. . . .

Level 2
. . . . . X . .
. . . . . . . .
X 0 . . 0 . 0 X
X 0 . . X . . .

Level 3
. 0 . . X . . . X . . 0
. 0 X . . . . . . . .
. . . . 0 . . . . 0 . .
. . . . X . X . . . X 0

=====
Game finished
Total turns: 27
Successful move count: 25
Winner: Player 1 (0)
=====
```

This example illustrates that the implemented environment can be inspected at the level of individual game trajectories, rather than only through aggregate statistics. The difference between total turns and successful move count also confirms the presence of forfeited moves caused by the stochastic realization rule. Although the displayed game uses base-

line agents for demonstration, the same watching interface can be applied to other policies as long as they provide a valid action for the current environment state.

6 Phase F: Interactive Game Interface

In addition to the terminal-based game watching script, this project also provides a lightweight interactive interface implemented with **Streamlit**. The purpose of this interface is to make the environment more directly observable and playable. Although a graphical interface is not required by the original assignment, it serves as a useful demonstration tool for inspecting the stochastic move mechanism, the agent interaction process, and the final game outcome.

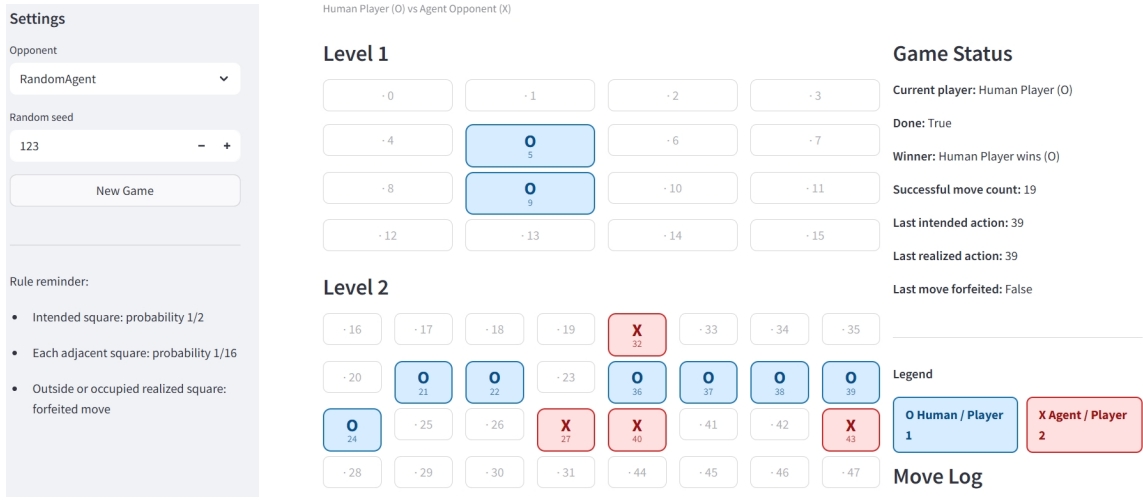


Figure 5: Lightweight interactive interface for playing the stochastic super tic-tac-toe game.

As shown in Figure 5, the interface is divided into three main regions. The left sidebar contains the game settings, including the opponent type and the random seed. The opponent can be selected as either `RandomAgent` or `HeuristicAgent`. The random seed controls the stochastic action realization process and makes the game behavior reproducible. A **New Game** button is also provided to reset the environment and start a new game. The sidebar further displays a short rule reminder: the intended square is realized with probability 1/2, each adjacent square is realized with probability 1/16, and a move is forfeited if the realized square is outside the board or already occupied.

The center panel displays the game board level by level. Each playable square is represented as a clickable cell with its corresponding action index. Empty cells are shown as light buttons marked with a dot, while occupied cells are displayed using different colors. In the current implementation, `O` represents the human player or Player 1, and `X` represents the agent opponent or Player 2. This design makes the board state more readable than the terminal rendering and allows the user to interact with the environment by directly clicking on a target square.

The right panel shows the current game status. It includes the current player, whether the game has terminated, the winner, the number of successful placements, the last

intended action, the last realized action, and whether the last move was forfeited. These quantities are important because the game is stochastic: the square selected by the user is only the intended action, while the final placement may be different due to random realization. Therefore, displaying both the intended and realized actions helps verify that the stochastic transition rule is being applied correctly.

The interface also includes a move log that records recent actions. Each entry reports the intended action, realized action, forfeiture status, and reward information. This allows users to inspect the trajectory of a game step by step. In the example shown in Figure 5, the game has already terminated, with the human player winning as Player 1. The interface shows that the last intended action and the last realized action are both 20, and the last move was not forfeited.

To play the game, the user first selects an opponent and random seed from the sidebar, then clicks **New Game**. The user plays as **O** and selects an empty square by clicking on the corresponding cell. After the human move is resolved by the stochastic environment, the selected agent opponent automatically makes its move as **X**. The interface then updates the board, status panel, and move log. This process repeats until either a player wins or the game ends in a draw.

Overall, the interactive interface provides an intuitive way to demonstrate that the implemented environment is not only usable for batch evaluation and DQN training, but also playable and inspectable at the level of individual decisions. It helps illustrate the key distinction between intended actions and realized actions, which is one of the central stochastic features of this project.

7 Conclusion

This project completes a full reinforcement learning workflow for a stochastic super tic-tac-toe game, including rule interpretation, environment implementation, baseline evaluation, and preliminary DQN training. Compared with standard tic-tac-toe, this task has a more complex board structure, a larger action space, stochastic move realization, and cross-level winning patterns. Therefore, the central challenge is not only to train an agent, but also to build a clear, stable, testable, and reproducible environment.

First, the game rules are formalized systematically. The board is modeled as a three-level structure with six 4×4 blocks and 96 playable squares. Logical coordinates, level-wise geometric coordinates, and stacked display coordinates are introduced to represent the board and enumerate winning patterns. Row, diagonal, and column wins are defined consistently and implemented through pre-enumerated winning lines.

Second, the `SuperTicTacToeEnv` environment is implemented. It supports reset, step execution, action legality checking, action masks, stochastic move realization, forfeited moves, winner detection, draw detection, and rendering. The stochastic action mechanism follows the project specification exactly: the intended target is realized with probability $1/2$, and each of the eight same-level adjacent candidates is realized with probability $1/16$. Invalid or occupied realized positions lead to forfeited moves. Systematic tests confirm the correctness of initialization, action masks, stochastic realization, boundary behavior, and winner detection.

Third, two baseline agents are implemented and evaluated. The `RandomAgent` serves as a minimal baseline, while the `HeuristicAgent` uses a one-step deterministic approximation to choose winning moves, block immediate threats, or otherwise play randomly. After correcting the player-perspective logic, the heuristic agent achieves a 96.0% win rate against the random agent over 200 games. This demonstrates that the environment can clearly reflect strategy quality.

Finally, a TorchRL-based DQN training pipeline is built. The two-player game is wrapped into a single-agent task against a fixed random opponent. The training pipeline uses a Q-network, target network, epsilon-greedy exploration with action masking, and a TorchRL `TensorDictReplayBuffer`. The DQN agent reaches intermediate win rates around 0.66–0.70, indicating that it captures some meaningful learning signals. However, the final performance remains unstable and does not consistently exceed the random first-player baseline or approach the heuristic baseline.

Overall, the project establishes a complete and extensible experimental framework for reinforcement learning in a stochastic board-game environment. Although the current DQN policy is not yet a strong strategy, the environment, baseline evaluations, and training pipeline provide a solid foundation for future work. Possible extensions include improved observation encoding, reward shaping, lower learning rates, slower exploration decay, longer training, more complete TorchRL-native components, stronger opponents, and self-play.